

5강 실습

데이터 정제 & 시각화

파트 1

결측치 처리

35분

파트 2

이상치 탐지

22분

파트 3

matplotlib

26분

파트 4

seaborn

20분

종합

파이프라인

20분

실습 안내 — 오늘 이렇게 합니다

📁 실습 파일 열기 → lecture5_practice.ipynb (Jupyter Notebook)

각 셀의 # TODO 주석 아래 코드를 작성하세요

📄 실습지

lecture5_practice.ipynb — 학생용 (코드 셀 비어있음)

✅ 정답

lecture5_practice_answer.ipynb — 완성 코드 포함

📁 데이터

lecture4_data/ 폴더 → sample_data.csv 사용

🔑 진행 방법

- 슬라이드에 제시된 문제와 기대 출력을 보고 직접 작성
- 모르면 힌트 함수 → 강의 자료 → 그다음 정답 파일 순서로
- 시간 안에 못 끝내도 괜찮습니다 — 정답 보고 이해하는 게 목표

🔧 환경 확인

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

print(pd.__version__) # 1.x 이상
print(sns.__version__) # 0.11 이상
```

데이터 로드 & 기본 확인

준비

🕒 5분

- ① pandas를 pd로 임포트하세요
- ② matplotlib.pyplot을 plt로 임포트하세요
- ③ seaborn을 sns로 임포트하세요
- ④ sample_data.csv 파일을 읽어 df에 저장하세요
- ⑤ df의 크기(shape)와 처음 3행(head)을 출력하세요

📄 기대 출력

(150, 5)

← df.shape

	이름	나이	점수	지역	등록일
0	김민준	25.0	92.0	서울	2023-01-05
1	이서연	NaN	85.0	경기	2023-01-08
2	박지훈	31.0	NaN	NaN	2023-01-12

💡 pd.read_csv() | df.shape | df.head(3)

🔗 결측치가 보이나요? → NaN 위치를 메모해 두세요. 파트 1에서 처리합니다.

파트 1

결측치 처리

`isnull` / `dropna` / `fillna`

1. 결측치 현황 파악

2. 결측치 시각화

3. 제거 (`dropna`)

4. 대체 (`fillna`)

5. 종합 미션

 총 35분

실습 1-1 — 결측치 현황 파악

1 - 1

어디에 얼마나 있나요?

🕒 7분

- ① `df.isnull().sum()` 으로 열별 결측치 개수를 출력하세요
- ② 결측치 비율(%)도 함께 계산해서 출력하세요
(결측 수 ÷ 전체 행 수 × 100)
- ③ `df.isnull().sum().sum()` 으로 총 결측치 수를 출력하세요

📎 기대 출력

이름	0
나이	5
점수	5
지역	0
등록일	0
dtype: int64	

나이	3.33 %
점수	3.33 %

총 결측치: 10

💡 `isnull().sum() | / len(df) * 100 | .round(2)`

🔍 추가 도전 (선택)

`df[ df.isnull().any(axis=1) ]` - 결측치가 있는 행만 골라서 출력해보세요

결측치가 있는 행만 보면 어떤 패턴이 보이나요?

실습 1-2 — 결측치 시각화

1 - 2

눈으로 직접 확인하기

🕒 7분

- ① seaborn heatmap으로 결측치 위치를 시각화하세요
 - yticklabels=False, cbar=False, cmap="viridis"
- ② 제목은 "결측치 위치 (노란색 = NaN)" 으로 설정하세요
- ③ plt.show()로 출력하세요

📄 기대 출력

이름	나이	점수	지역	등록일

노란색 = NaN

💡 `sns.heatmap(df.isnull(), ...) | plt.title() | plt.show()`

```
import seaborn as sns
import matplotlib.pyplot as plt

# TODO: heatmap으로 결측치 시각화
sns.heatmap( ____ )
plt.title( ____ )
plt.show()
```

실습 1-3 — 결측치 제거 (dropna)

1 - 3

결측치 있는 행을 제거하기

🕒 7분

- ① `df.dropna()`로 결측치가 있는 행을 모두 제거하고 `df_drop`에 저장하세요
- ② "점수" 열에만 결측치가 있는 행을 제거하고 `df_drop2`에 저장하세요 (`subset` 파라미터 사용)
- ③ 원본 `df`, `df_drop`, `df_drop2`의 `shape`를 비교하세요

📄 기대 출력

원본: (150, 5)

`df_drop`: (140, 5) ← 10행 제거

`df_drop2`: (145, 5) ← 점수 결측 5행만 제거

💡 `df.dropna()` | `df.dropna(subset=["점수"])` | `.shape`

```
# TODO: 결측치 제거
df_drop = df.__( )
df_drop2 = df.__(subset=__)

print("원본: ", df.___)
print("df_drop:", df_drop.___)
print("df_drop2:", df_drop2.___)
```

실습 1-4 — 결측치 대체 (fillna)

1 - 4

결측치를 값으로 채우기

🕒 7분

원본 보호용으로 `df_fill = df.copy()` 를 먼저 만드세요

- ① "나이" 열의 결측치를 나이 열의 평균으로 채우세요
- ② "점수" 열의 결측치를 점수 열의 중앙값으로 채우세요
- ③ `df_fill.isnull().sum()` 으로 결측치가 0이 됐는지 확인하세요

📄 기대 출력

처리 전

나이 5

점수 5

처리 후

이름 0

나이 0

← 평균으로 채움

점수 0

← 중앙값으로 채움

지역 0

등록일 0

💡 `df.copy()` | `.mean()` | `.median()` | `fillna(값, inplace=True)`

```
df_fill = df.copy()
# TODO: 나이 → 평균으로 채우기
df_fill["나이"].fillna( ____, inplace=True)
# TODO: 점수 → 중앙값으로 채우기
df_fill["점수"].fillna( ____, inplace=True)
# TODO: 결측치 확인
print(df_fill.isnull().sum())
```

파트 1 종합 미션

★ 미션

🕒 7분

📋 미션: mission_data.csv 결측치 완전 정제

```
# 1. mission_data.csv 읽기 (인코딩: "euc-kr")
df_m = pd.read_csv( ____, encoding= ____ )

# 2. 결측치 현황 출력
print(df_m.isnull().sum())

# 3. 수치형 열 → 중앙값으로 대체 (반복문 활용)
for col in df_m.select_dtypes("number").columns:
    df_m[col].fillna( ____, inplace=True)

# 4. 범주형 열 → 최빈값으로 대체
for col in df_m.select_dtypes("object").columns:
    df_m[col].fillna( ____, inplace=True)

# 5. 처리 후 검증 - 모든 열이 0이어야 합니다
print("결측치 합계:", df_m.isnull().sum().sum())
```

💡 select_dtypes("number")는 수치형 열만 골라줍니다 | mode()[0] → 최빈값 첫 번째 값

파트 2

이상치 탐지

boxplot / IQR / clip

1. 시각적 확인 (boxplot)

2. IQR로 탐지

3. clip으로 처리

4. 종합 미션

 총 22분

실습 2-1 — boxplot으로 이상치 확인

2 - 1

눈으로 먼저 이상치 파악하기

🕒 6분

정제된 df_fill 데이터를 사용하세요

- ① "점수" 열 boxplot을 그리세요 (seaborn)
- ② 제목: "점수 분포 및 이상치 확인"
- ③ 이상치(점)가 보이나요? 최대/최솟값을 출력해보세요
df_fill["점수"].describe() 로 확인

📄 기대 출력

count	145.000000	
mean	82.413793	
std	8.225...	
min	55.000000	← 이상치?
25%	77.000000	
50%	83.000000	
75%	89.000000	
max	120.000000	← 이상치?

💡 sns.boxplot(data=df_fill, y="점수") | plt.title() | .describe()

```
# TODO: 점수 열 boxplot
sns.boxplot(data= ____, y= ____ )
plt.title( ____ )
plt.show()
```

```
# TODO: 기술통계 확인
print(df_fill["점수"].describe())
```

실습 2-2 — IQR로 이상치 탐지

2 - 2

수식으로 이상치 경계 계산하기

🕒 8분

- ① Q1, Q3, IQR을 계산하세요 (quantile 사용)
- ② 하한 경계 = $Q1 - 1.5 \times IQR$
상한 경계 = $Q3 + 1.5 \times IQR$
- ③ 이상치 행을 골라 outliers 에 저장하세요
- ④ 이상치 개수와 이상치 행을 출력하세요

📄 기대 출력

Q1 = 77.0 Q3 = 89.0
IQR = 12.0
하한 경계 = 59.0
상한 경계 = 107.0

이상치 개수: 3

	이름	나이	점수
5	홍길동	45.0	120.0
38	김철수	22.0	55.0

💡 `.quantile(0.25) | .quantile(0.75) | df[(조건1) | (조건2)]`

```
Q1 = df_fill["점수"].quantile( ____ )
Q3 = df_fill["점수"].quantile( ____ )
IQR = ____ - ____
lower = Q1 - ____ * IQR
upper = Q3 + ____ * IQR
outliers = df_fill[(df_fill["점수"] < ____) |
                    (df_fill["점수"] > ____)]
print(f"이상치 개수: {len(outliers)}")
print(outliers)
```

실습 2-3 — clip으로 이상치 처리 & 전후 비교

2 - 3

경계값으로 대체하고 결과 확인하기

🕒 8분

- ① df_fill 을 복사해 df_clean 에 저장하세요
- ② df_clean의 "점수" 열에 clip(lower, upper) 적용하세요
- ③ 서브플롯 1행 2열로 전/후 boxplot을 나란히 그리세요
 - 왼쪽: 처리 전 (df_fill)
 - 오른쪽: 처리 후 (df_clean)
- ④ plt.tight_layout() 후 출력하세요

📎 기대 출력

처리 전 최대값: 120.0
처리 후 최대값: 107.0 ← 상한 경계

[두 boxplot이 나란히 표시됨]
왼쪽: 점 (이상치) 보임
오른쪽: 깨끗하게 정리됨

💡 df.copy() | .clip(lower, upper) | plt.subplots(1, 2) | ax=axes[0]

```
df_clean = df_fill.copy()
df_clean["점수"] = df_clean["점수"].clip( ____, ____ )
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
sns.boxplot(data=df_fill, y="점수", ax=axes[_])
sns.boxplot(data=df_clean, y="점수", ax=axes[_])
axes[0].set_title("처리 전")
axes[1].set_title("처리 후")
plt.tight_layout()
plt.show()
```

파트 3

matplotlib

시각화

plot / bar / scatter / hist / subplots

1. 선 그래프

2. 막대 그래프

3. 산점도

4. 서브플롯 + 저장

 총 26분

실습 3-1 — 선 그래프 (plt.plot)

3 - 1

시간 추이 데이터 그리기

🕒 6분

주어진 월별 매출 데이터로 선 그래프를 그리세요

- ① plt.figure(figsize=(8, 4)) 로 크기 설정
- ② plt.plot() 으로 선 그래프 그리기
 - color="steelblue", linewidth=2, marker="o"
- ③ 제목, X축, Y축 레이블 추가
- ④ plt.grid(True) 로 격자 추가 후 출력

📎 기대 출력

데이터

월 = [1, 2, 3, 4, 5, 6]

매출 = [120, 145, 132, 178, 155, 192]

→ x축: 월, y축: 매출 (만원)

→ 파란 선 + 점 마커

→ 격자 있음

💡 plt.plot(x, y, color=, linewidth=, marker=) | plt.xlabel() | plt.ylabel()

```
월 = [1, 2, 3, 4, 5, 6]
매출 = [120, 145, 132, 178, 155, 192]
plt.figure(figsize=(8, 4))
plt.plot( ____, ____, color=____, linewidth=____, marker=____)
plt.title("월별 매출 추이")
plt.xlabel( ____ )
plt.ylabel("매출 (만원) ")
plt.grid( ____ )
plt.show()
```

실습 3-2 — 막대 그래프 (plt.bar)

3 - 2

그룹별 비교 차트 그리기

🕒 6분

df_clean 데이터로 지역별 평균 점수 막대 그래프를 그리세요

- ① `groupby("지역")["점수"].mean()` 으로 지역별 평균 계산
- ② 결과를 `region_mean` 변수에 저장하세요
- ③ `plt.bar()` 로 막대 그래프 그리기
- ④ 제목: "지역별 평균 점수"

📄 기대 출력

지역	
경기	81.3
경남	83.7
경북	80.1
부산	84.2
서울	82.9
...	

- 지역 이름이 x축
- 평균 점수가 막대 높이

💡 `.groupby().mean()` | `plt.bar(index, values)` | `region_mean.index` | `.values`

```
region_mean = df_clean.groupby( ____ )[ ____ ].mean()
plt.figure(figsize=(9, 4))
plt.bar(region_mean.____ , region_mean.____ ,
        color='steelblue', edgecolor='white')
plt.title("지역별 평균 점수")
plt.xlabel("지역")
plt.ylabel("평균 점수")
plt.xticks(rotation=30)
plt.show()
```

실습 3-3 — 산점도 (plt.scatter)

3 - 3

두 변수의 관계 파악하기

🕒 6분

df_clean 으로 나이 vs 점수 산점도를 그리세요

- ① X축: 나이, Y축: 점수
- ② color="orange", alpha=0.6, s=60
- ③ 제목: "나리와 점수의 관계"

★ 추가 도전: 나이가 30 이상인 점만 빨간색으로 구분해서 그리세요
(두 번 scatter 호출, label 파라미터 활용)

📎 기대 출력

- 나이 분포: 20~55세
- 점수 분포: 59~107점
- 산점도에 패턴이 보이나요?
(양의 상관? 음의 상관?)

★ 도전 성공 시:

- 주황 점: 30세 미만
- 빨간 점: 30세 이상

💡 plt.scatter(x, y, color=, alpha=, s=, label=) | plt.legend()

```
plt.figure(figsize=(8, 5))
plt.scatter(df_clean["나이"], df_clean["점수"],
            color=__, alpha=__, s=__)
plt.title("나리와 점수의 관계")
plt.xlabel("나이")
plt.ylabel("점수")
plt.show()
```

실습 3-4 — 서브플롯 & 차트 저장

3 - 4

여러 차트 한 화면에 + 파일로 저장

🕒 8분

1행 3열 서브플롯을 만들고 세 가지 차트를 배치하세요

- ① `axes[0]` → 점수 히스토그램 (`bins=20`)
- ② `axes[1]` → 나이 vs 점수 산점도
- ③ `axes[2]` → 지역별 인원 막대 그래프
(`value_counts()` 활용)
- ④ `plt.tight_layout()` 후 `chart.png` 로 저장 (`dpi=150`)

📎 기대 출력

→ 3칸짜리 가로 차트

[히스토그램] [산점도] [막대그래프]

→ `chart.png` 저장 완료
현재 폴더에서 파일 확인!

💡 `plt.subplots(1, 3, figsize=(15,4))` | `ax=axes[N]` | `plt.savefig("chart.png", dpi=150)`

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
df_clean["점수"].hist(ax=axes[_], bins=20)
axes[0].set_title("점수 분포")
axes[_].scatter(df_clean["나이"], df_clean["점수"])
axes[1].set_title("나이 vs 점수")
counts = df_clean["지역"].value_counts()
axes[_].bar(counts.index, counts.values)
axes[2].set_title("지역별 인원")
plt.tight_layout()
plt.savefig("chart.png", dpi=___)
plt.show()
```

파트 4

seaborn

시각화

histplot / boxplot / heatmap / regplot

1. histplot

2. 상관관계 heatmap

3. regplot (회귀선)

 총 20분

실습 4-1 — histplot & boxplot 비교

4 - 1

분포를 두 가지 방법으로 보기

🕒 7분

`sns.set_theme(style="whitegrid")` 설정 후

1행 2열 서브플롯을 만드세요

① `axes[0] → sns.histplot("점수", kde=True)`

제목: "점수 분포 + 곡선"

② `axes[1] → sns.boxplot(x="지역", y="점수")`

`xticks rotation=30` 추가

제목: "지역별 점수 분포"

📎 기대 출력

→ 왼쪽: 히스토그램 + KDE 곡선
점수 분포 모양이 보임

→ 오른쪽: 지역별 박스플롯
각 지역의 중앙값과
이상치 여부가 한눈에 보임

💡 `sns.histplot(data=, x=, kde=True, ax=)` | `sns.boxplot(data=, x=, y=, ax=)`

```
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(data=df_clean, x=__, kde=__, ax=axes[_])
axes[0].set_title("점수 분포 + 곡선")
sns.boxplot(data=df_clean, x=__, y=__, ax=axes[_])
axes[1].set_title("지역별 점수 분포")
plt.xticks(rotation=__)
plt.tight_layout()
plt.show()
```

실습 4-2 — 상관관계 히트맵 (heatmap)

4 - 2

변수 간 상관관계 시각화하기

🕒 7분

- ① df_clean의 수치형 열만 골라 corr() 로 상관행렬 계산
- ② sns.heatmap()으로 시각화
 - annot=True → 수치 표시
 - fmt=".2f" → 소수점 2자리
 - cmap="coolwarm" → 색상
- ③ 제목: "변수 간 상관관계"
- ④ 가장 높은 상관관계 쌍은 어디인가요?

📄 기대 출력

	나이	점수
나이	1.00	0.23
점수	0.23	1.00

- 히트맵: 빨강=양의 상관
파랑=음의 상관
- 대각선은 항상 1.0
(자기 자신과의 상관)

💡 `.select_dtypes("number").corr()` | `sns.heatmap(corr, annot=True, fmt=, cmap=)`

```
corr = df_clean.select_dtypes(__).corr()

plt.figure(figsize=(6, 4))
sns.heatmap(corr,
            annot=__,
            fmt=__,
            cmap=__)
plt.title("변수 간 상관관계")
plt.show()
```

실습 4-3 — 산점도 + 추세선 (regplot)

4 - 3

관계와 추세를 한 번에 보기

🕒 6분

나이와 점수의 관계를 추세선과 함께 시각화하세요

- ① sns.regplot() 사용
- ② scatter_kws={"alpha": 0.5} → 점 투명도
- ③ line_kws={"color": "red"} → 추세선 빨간색
- ④ 제목: "나이 vs 점수 + 추세선"

★ 추세선의 기울기로 상관 방향을 읽어보세요

📄 기대 출력

- 산점도 + 빨간 추세선
- 회색 음영 = 신뢰 구간

추세선이 올라가면? 양의 상관
추세선이 내려가면? 음의 상관

★ 나이가 늘수록 점수가
높아지는 경향이 있나요?

💡 `sns.regplot(data=, x=, y=, scatter_kws={"alpha":.}, line_kws={"color":.})`

```
plt.figure(figsize=(7, 5))
sns.regplot(data=df_clean, x=__, y=__,
            scatter_kws={"alpha": ____},
            line_kws={"color": ____})
plt.title("나이 vs 점수 + 추세선")
plt.show()
```

★ 종합 미션 — 전체 파이프라인

★★ 종합

🕒 20분

처음부터 끝까지 혼자 완성이기 — 데이터 읽기 → 정제 → 시각화 → 저장

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ① 파일 읽기 (sample_data.csv)
df = ____

# ② 결측치 처리 — 수치형: 중앙값, 범주형: 최빈값
for col in df.select_dtypes("number").columns:
    df[col].fillna(____, inplace=True)

# ③ 이상치 처리 (점수 열, IQR + clip)
Q1, Q3 = df["점수"].quantile([0.25, 0.75])
IQR = ____
df["점수"] = df["점수"].clip(____, ____ )

# ④ 시각화 — 3개 서브플롯
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sns.histplot(data=df, x="점수", kde=True, ax=____)
axes[0].set_title("점수 분포")
sns.boxplot(data=df, x="지역", y="점수", ax=____, palette="Set2")
axes[1].set_title("지역별 점수")
corr = df.select_dtypes("number").corr()
sns.heatmap(corr, annot=True, ax=____, cmap="coolwarm")
axes[2].set_title("상관관계")
plt.tight_layout()

# ⑤ 저장 (pipeline_result.png, dpi=150)
plt.savefig(____, dpi=____)
plt.show()
print("✅ 완료!")
```

도전 미션 — 시간이 남는다면!

★★★ 도전

도전 1 — 히스토그램 비교

지역이 "서울"인 그룹과 나머지 그룹의 점수 분포를
같은 화면에 histplot으로 겹쳐 그리세요
(alpha=0.5, label 파라미터 활용)

도전 2 — pair plot

sns.pairplot(df[["나이", "점수"]]) 을 그려보세요
모든 변수 쌍의 관계를 한 번에 볼 수 있습니다

도전 3 — 정제 함수 만들기

```
def clean_data(df):  
    # 결측치 + 이상치 처리 코드  
    return df_clean
```

위 함수를 완성하고 새 파일을 인수로 넣어 테스트하세요

💡 끝냈다면? → 정답 파일(lecture5_practice_answer.ipynb)과 비교해보세요
코드가 달라도 결과가 같으면 정답입니다! 다양한 방법이 있습니다.

5강 실습 마무리

□ 결측치 처리

- `isnull().sum()` 확인 → `dropna` / `fillna` 처리
- 처리 후 `sum().sum() = 0` 으로 검증

⚠ 이상치 탐지

- `boxplot`으로 시각 확인
- IQR 계산 → `clip(lower, upper)` 적용

📊 matplotlib

- `plot` / `bar` / `scatter` / `hist`
- `subplots()` + `savefig()`

🍷 seaborn

- `histplot(kde)` / `boxplot` / `heatmap` / `regplot`
- `set_theme()`으로 스타일 설정

💡 정제 → 시각화 파이프라인을 직접 완성했습니다. 이제 새 데이터에도 똑같이 적용할 수 있습니다!

🔑 오늘 배운 핵심 코드 패턴

```
df.isnull().sum()          # 결측치 확인
df["열"].fillna(df["열"].median(), inplace=True) # 결측치 대체
df["열"] = df["열"].clip(lower, upper)          # 이상치 처리
sns.heatmap(df.corr(), annot=True)              # 상관관계
```